Módulo 08 — Deploy e operação

Cada slide da apresentação explicado em linguagem simples: termos, o que falar e o que responder.

---

1. Slide a slide

Fonte: https://github.com/MateusCastro2203/Mentoria-AI

Slide a slide

Este guia explica cada slide da apresentação em linguagem simples: o que significa cada termo, o que falar e o que responder se perguntarem. Leia uma vez inteiro antes da aula; durante a aula, as notas do PPTX bastam.

> **Se só der tempo de entender uma coisa:** em produção, o curador roda **sozinho** (um "cron" dispara toda segunda) num **Cloudflare Worker**, um pedaço de código que roda na nuvem da Cloudflare. Como ninguém está olhando, três coisas passam a ser obrigatórias: **ver** o que aconteceu (o rastro: quanto custou, onde demorou), **desfazer** quando der errado (versões do prompt com botão de voltar) e **aguentar** os imprevistos (rodar duas vezes, o provedor cair).

Antes da aula (uns 10 minutos, quase tudo esperando o modelo):

1. Uma vez: `cp projeto-final/.dev.vars.example projeto-final/.dev.vars` (cria o "token do editor" local).
2. Terminal 1: `pnpm -F @mentoria/curador m08:dev:solucao`. Sobe o Worker **na sua máquina** (não precisa de conta na Cloudflare). Deixe aberto.
3. Terminal 2: `pnpm -F @mentoria/curador m08:operar rodar`. Simula o cron: o curador roda com 8 notícias (uns 4 minutos) e para numa pausa.
4. `pnpm -F @mentoria/curador m08:avaliar:solucao` (uns 1,5 minuto) e `pnpm -F @mentoria/curador m08:operar promover v1`. Deixa a versão v1 do prompt ativa.
5. Para a demo do portão (slide 13), prepare uma "v2" piorada, como está em Preparar a v2, e rode a avaliação dela antes.
6. `pnpm -F @mentoria/ex08-operacao demo:falhas:solucao` (instantânea): rode uma vez para ver a saída.

Se algo der errado ao vivo, os números de todos os slides vieram dessas mesmas execuções; dá para mostrar só os slides.

Slide 1 — Capa

Diga: *"Até hoje, o curador rodava no seu terminal, quando você mandava. Hoje ele vai rodar sozinho, toda segunda, num lugar que você não vê. E aí as perguntas mudam."*

Slide 2 — Hoje

Os objetivos. Não leia a lista. Avise que é a última aula e que os 15 minutos de discussão terminam numa retrospectiva da mentoria.

Slide 3 — A frase da aula

> Em produção, o que importa é o que você consegue ver e desfazer.

- **Ver:** saber, depois do fato, o que o sistema fez, quanto custou e onde demorou.
- **Desfazer:** ter um jeito rápido de voltar para a versão anterior.

A segunda linha é uma atitude: escreva o código **supondo o pior**. O agendador pode disparar duas vezes, o provedor pode cair no meio, e a conta pode vir maior do que você estimou.

Slide 4 — Divisória "1 · Empacotar e escolher a plataforma"

Empacotar = juntar o código e tudo de que ele precisa num pacote que roda em outro lugar.

Slide 5 — Onde o curador pode morar

As três plataformas do curso:

- **Cloudflare Workers:** roda JavaScript/TypeScript na nuvem da Cloudflare, sem você cuidar de servidor. Tem **Cron Triggers** (agendamento, como um despertador: "toda segunda às 9h") e armazenamento (**KV**, um dicionário chave → valor na nuvem; **D1**, um banco SQLite).
- **Google Apps Script:** roda scripts dentro do Google Workspace. Tem agendamento, mas não roda as bibliotecas npm que usamos.
- **Google Sites:** páginas estáticas. Não roda código no servidor.

Por isso o curador vai para o Workers. Apps Script e Sites continuam úteis, por exemplo, para uma página que mostra a última edição.

Sem disco: no Workers não existe pasta de arquivos. Os feeds RSS e a skill entram **dentro do pacote**, como texto.

Variável × segredo: variável é configuração que muda por ambiente (endereço do modelo, quantas notícias). **Segredo** é o que não pode vazar (o token do editor, a chave de um provedor): vai por `wrangler secret put`, nunca no código.

Se perguntarem "por que não um servidor comum?": dá, mas aí você cuida de máquina, atualização e disponibilidade. No Workers, você só sobe o código.

Slide 6 — Os limites viram requisito

Toda plataforma tem limites. O ponto do slide: **os limites mudaram o desenho do curador**.

- **50 chamadas externas por execução** (plano gratuito). Cada chamada ao modelo conta. Com 32 notícias seriam umas 70 chamadas, então a versão agendada usa **8 notícias**.
- **10 ms de CPU** (gratuito). CPU é o tempo em que o processador trabalha **de fato**. Esperar a resposta do modelo não conta, mas montar o grafo conta. Não medi se cabe; isso só se vê publicando.
- **15 minutos** de duração máxima de um cron.
- **KV: 1 escrita por segundo na mesma chave.** O grafo salva o checkpoint várias vezes por segundo; por isso gravamos só no fim.

Coletor virou código: na Aula 4, o coletor era um agente que escolhia as fontes. Na versão agendada, ele virou código simples (pega as primeiras notícias), porque às 9h de segunda ninguém está olhando o agente, e cada passo dele gasta uma das 50 chamadas.

Curiosidade, se houver tempo: rodar o LangGraph no Workers exigiu dois ajustes que só apareceram testando de verdade. Um deles: sem uma configuração de 3 linhas, o `interrupt()` quebrava com "Called interrupt() outside the context of a graph".

Slide 7 — Divisória "2 · Rastreio, custo e latência"

Rastreio (*tracing*): registrar cada passo de uma execução para poder investigar depois.

Slide 8 — Um evento por chamada ao modelo

Todas as chamadas ao modelo no curso passam por um único pacote (`@mentoria/llm`). Então é lá que medimos: cada chamada vira um **evento** com tokens, tempo e o contexto (qual papel, qual notícia).

- `observarChamadas`: "me avise de cada chamada".
- `comAtributos`: "tudo o que acontecer aqui dentro é do papel *redigir*, notícia *n13*".

AsyncLocalStorage é o mecanismo do Node (e do Workers) que carrega esse contexto através das esperas (`await`). Sem ele, 8 classificações rodando ao mesmo tempo se misturariam.

Se perguntarem "existe pronto?": sim. O padrão de mercado é o **OpenTelemetry**, um formato aberto de rastros que as ferramentas de observabilidade entendem, e o AI SDK já emite nesse formato. O nosso é caseiro de propósito: dá para ver as peças e testar sem internet.

Slide 9 — O rastro real de uma edição

A tabela veio de uma execução de verdade (8 notícias, modelo local).

- **p50** (mediana): metade das chamadas foi mais rápida que isso.
- **p95**: 95% foram mais rápidas. Mostra os casos ruins.

A surpresa: o **revisor**, que só responde "aprovado ou não, e por quê", gerou **87%** dos tokens de saída. Investigando: quando pedimos a resposta num formato JSON fixo (*JSON schema*), o Ollama contou 758 tokens para uma resposta de 326 caracteres; sem o formato fixo, 152 tokens para 538 caracteres. **Não investiguei a causa**, e diga isso. A lição é: custo se **mede**, não se adivinha. Sem o rastro, ninguém olharia para o revisor.

Pergunte: *"O que vocês fariam com o revisor?"* (Respostas boas: pedir um motivo curto; trocar por um modelo menor; trocar parte dele por regras; medir antes e depois com a eval.)

Slide 10 — Custo e latência: como ler

Custo = tokens × preço. Os provedores cobram por milhão de tokens, com preços diferentes para entrada e saída (veja a página de preços do provedor).

- **Estimar** (contar caracteres e dividir por 4) serve para um **teto** antes de chamar.
- **Medir** (o provedor devolve quantos tokens usou) serve para a **conta**.
- **Modelo local:** não tem preço por token, mas tem custo de tempo e de máquina.

Soma ≠ duração: as chamadas rodam em paralelo. Somando o tempo de todas dá 829 s, mas a execução inteira levou 245 s. E, num modelo local, as chamadas em paralelo esperam na fila, o que aumenta o p95.

Analogia: num restaurante com 4 cozinheiros, somar o tempo de preparo de todos os pratos não diz quanto tempo o cliente esperou.

Slide 11 — Divisória "3 · Versão do prompt e rollback"

Rollback = voltar para a versão anterior.

Slide 12 — O prompt é código que muda o comportamento

Mudar o prompt muda o que o sistema faz, então ele merece o mesmo cuidado que o código:

1. **Versão imutável com hash.** O **hash** é uma "impressão digital" do texto: mudou uma vírgula, muda o hash. Assim, a avaliação fica presa ao texto que foi avaliado.
2. **Portão antes de ativar**, em duas camadas:
    - **absoluto**, no CI: nota mínima (macro-F1 ≥ 0,8, a métrica da Aula 3) e nenhum caso perigoso publicado;
    - **relativo**, no Worker: não pode ser pior que a versão **que está no ar**. Só o Worker sabe qual é.
3. **Cada execução registra a versão usada.** Se uma edição sair ruim, dá para saber qual prompt a gerou.
4. **Rollback sem deploy:** a versão ativa é só um "ponteiro" guardado no KV. Voltar é mudar o ponteiro.

O modelo também é versão: trocar o modelo muda tudo. Por isso a comparação só vale entre avaliações do mesmo modelo.

Slide 13 — O portão pegou uma regressão de verdade

Três versões, avaliadas de verdade:

- **v1:** a skill da Aula 4. Nota 0,967, ativada.
- **v2:** alguém quis mais itens por semana e afrouxou o critério ("considere relevante tudo que mencione IA"). Nota 0,852: **passa** no mínimo do CI (0,8), mas o Worker **barrou** por ser pior que a v1. É o caso típico de regressão: uma mudança que parece boa piora o resultado.
- **v1.1:** mudança mínima, mesma nota. Ativada e depois **revertida**, para mostrar o rollback.

Ao vivo: `pnpm -F @mentoria/curador m08:operar promover v2` (sai erro 422 com o motivo) e depois `reverter`.

Preparar a v2 (para o slide 13)

```
mkdir -p projeto-final/saidas/skill-v2
cp -r projeto-final/solucao/skills/curar-noticia projeto-final/saidas/skill-v2/
# edite projeto-final/saidas/skill-v2/curar-noticia/SKILL.md: troque o "Critério de
relevância" por
#   "Relevante: qualquer notícia que mencione IA. A newsletter precisa de mais itens
por semana."
pnpm -F @mentoria/curador m08:avaliar:solucao -- --pasta saidas/skill-v2/curar-noticia
```

O `m08:avaliar` guarda só a última avaliação: avalie a v2 logo antes de promovê-la.

Slide 14 — O que o rollback não desfaz

- **Volta:** o prompt (o ponteiro) e o código (o Workers guarda as últimas 100 versões publicadas).
- **Não volta:** o que já saiu para o mundo. Dados gravados, a edição publicada, o e-mail enviado.

Implantação gradual: em vez de trocar tudo de uma vez, mandar só uma parte do tráfego (ex.: 10%) para a versão nova e observar. No prompt, isso se chama **canário** (do canário que mineiros levavam para detectar gás antes de todo mundo). É o desafio do exercício.

Ligue com a Aula 7: é por isso que a pausa antes de publicar continua importante. Ela segura justamente o que não volta.

Slide 15 — Divisória "4 · Evals em CI"

CI (*integração contínua*): a cada mudança enviada ao repositório, um serviço (aqui, o GitHub Actions) roda as verificações automaticamente.

Slide 16 — Duas velocidades

- **Testes offline:** rodam em todo envio, em segundos, com um modelo simulado. Pegam código quebrado.
- **Eval com o modelo de verdade:** só quando o prompt, a eval ou os dados mudam. Leva minutos (o modelo roda no processador do servidor do GitHub): medi ~6 min no total, dos quais 257 s de eval, com o mesmo resultado do local (macro-F1 0,967). Pega quando o prompt piorou.

Se a eval reprova no portão, o PR (pedido de mudança) fica **vermelho** e não deveria entrar.

Se perguntarem "por que não rodar a eval sempre?": ela é lenta e custa. E um aviso que demora e às vezes falha por acaso vira um aviso que o time ignora.

Ao vivo: abra os arquivos `.github/workflows/ci.yml` e `eval-prompt.yml` no GitHub e, se houver, uma execução na aba *Actions*.

Slide 17 — Divisória "5 · Demo: o cron no Worker"

A demo usa o Worker rodando na sua máquina (`wrangler dev`). É o mesmo código que iria para a nuvem.

Slide 18 — O curador como Worker

- **Cron → `scheduled()`:** o agendador chama o Worker, que roda o grafo da Aula 7 até terminar ou pausar.
- **Revisão por HTTP:** na Aula 7, você respondia as pausas no terminal. Agora a "tela" é o endereço web do Worker (`/pendentes`, `/retomar`). **O grafo não mudou**, como a Aula 7 prometeu.
- **Token:** as rotas que mudam algo pedem um token. Uma rota de aprovação aberta seria uma porta para qualquer um publicar.

Ao vivo:

1. `pnpm -F @mentoria/curador m08:operar pendentes`: mostra a pausa `n39` (notícia sem fonte).
2. `pnpm -F @mentoria/curador m08:operar responder '{"n39":{"acao":"rejeitar","motivo":"sem fonte"}}'`: aparece a pausa da edição inteira.
3. `pnpm -F @mentoria/curador m08:operar responder '{"edicao":{"acao":"aprovar"}}'`: publicada.
4. `pnpm -F @mentoria/curador m08:operar edicao`: o markdown publicado.
5. `pnpm -F @mentoria/curador m08:operar rodar` de novo: no terminal do Worker aparece `edicao-pulada`, porque já rodou hoje. Isso é **idempotência**: fazer de novo não muda nada. (A chave é o **dia**: se você preparou a demo num dia e dá a aula no outro, esse comando começa uma edição nova, de uns 4 minutos. Nesse caso, só explique.)

Slide 19 — Um job que aguenta produção

É o exercício. Cada linha é uma situação real:

- **Cron disparou duas vezes:** acontece (reinício, alguém clicou em "executar"). **Idempotência:** antes de tudo, ver se o dia já deu certo.
- **429 / 503:** 429 = "muitos pedidos, espere"; 503 = "serviço fora do ar". São **transitórios**, então vale tentar de novo, esperando cada vez mais (**backoff**: 1 s, 2 s, 4 s).
- **Resposta fora do formato:** tentar de novo não resolve e custa.
- **Entrada enorme:** alguém colou um log gigante num chamado. O **teto de custo** impede que isso vire uma fatura enorme.

Retry numa camada só: o AI SDK já tenta de novo sozinho. Se o seu código também tenta, as tentativas se multiplicam. O artigo da Amazon do slide dá um exemplo em que a carga no banco cresce 243 vezes.

Se sobrar tempo: `pnpm -F @mentoria/ex08-operacao demo:falhas:solucao` mostra as cenas em segundos.

Slide 20 — Divisória "Discussão"

15 minutos. Guarde pelo menos 5 para a retrospectiva (pergunta 5).

Slide 21 — Para conversar (respostas esperadas)

1. **Job agendado no seu trabalho; e se rodar duas vezes?** Exemplos: relatório diário, triagem de e-mails, conciliação. Se rodar duas vezes: mensagem duplicada, cobrança dupla, registro repetido. A resposta boa traz uma **chave de idempotência** (o dia, o id do pedido).
2. **Quem promove uma versão de prompt?** Resposta madura: **os dois**. O CI garante o mínimo (não piorou na eval); uma pessoa decide se a mudança faz sentido para o negócio. Cuidado com "o CI aprovou, então está bom": a eval só mede o que está no dataset.
3. **O revisor custa 87% dos tokens de saída.** Opções: pedir um motivo curto; trocar por um modelo menor; trocar a parte simples por regras (tamanho, número de frases); amostrar (revisar só parte). **Como saber se a qualidade caiu:** rodar a eval de resumos (Aula 3) antes e depois. Decisão sem medir é chute.
4. **O que não volta com rollback?** Mensagens enviadas, pagamentos, dados apagados, edições publicadas, decisões que alguém já tomou com base no resultado. O que não volta precisa de **aprovação antes** (Aula 7), não de rollback depois.
5. **Retrospectiva.** Não há resposta certa. Anote o que aparecer: ajuda a melhorar a próxima turma.

Slide 22 — Para fechar

- **Exercício:** o job diário com idempotência, retry, orçamento e custo. Desafio: o canário.
- **Projeto final M8:** o curador no Worker, com as versões do prompt.
- **Leitura extra:** monitoramento em produção (alertas, *drift*), na apostila. *Drift* = o mundo ou o modelo mudam por fora do seu código, e o sistema piora sem ninguém mexer nele.

Slide 23 — Encerramento

Agradeça. Lembre que todo o material fica no repositório, com as soluções e as leituras extras, e que dá para refazer qualquer aula sozinho.

Glossário rápido

Termo	Em uma frase
Cloudflare Worker	código que roda na nuvem da Cloudflare sem você cuidar de servidor
Cron / Cron Trigger	agendamento ("toda segunda às 9h"); o nome vem de um agendador clássico do Unix
KV	armazenamento chave → valor na nuvem da Cloudflare
`wrangler`	a ferramenta de linha de comando da Cloudflare para rodar e publicar Workers
Bundle / empacotar	juntar o código e o que ele precisa num pacote só
Segredo	configuração que não pode vazar (token, chave); nunca vai no código
Subrequest	uma chamada que o Worker faz para fora (cada chamada ao modelo é uma)
Rastro (*trace*)	o registro de uma execução, passo a passo
p50 / p95	tempo abaixo do qual ficam 50% / 95% das chamadas
Hash	"impressão digital" de um texto; muda se o texto muda
Portão (*gate*)	regra que precisa passar para a mudança entrar
Rollback	voltar para a versão anterior
Canário	mandar só uma parte das execuções para a versão nova e observar
CI	verificações automáticas a cada mudança no repositório
Idempotência	rodar de novo não muda o resultado nem repete o efeito

Termo	Em uma frase
Retry / backoff	tentar de novo / esperar cada vez mais entre as tentativas
429 / 503	"muitos pedidos, espere" / "serviço indisponível"
Drift	o sistema piora porque o mundo ou o modelo mudaram por fora do código